科目：データベースの基礎

第3回 リレーショナルデータモデル：意味記述

講師：鈴木源吾

前回の復習

1. リレーショナルデータベースについて
    - リレーショナルデータモデルに従って作られたデータベース．SQL言語が使える
2. リレーショナルデータモデルの構造
    - リレーション・タプル・属性・リレーションスキーマ・次数・ドメイン
    - リレーションは集合である（タプルの順序に意味はない，同じタプルは2つない）
3. 第1正規形
    - 各属性のドメインがシンプル．組は属性に分け，集合はタプルに分けて正規化する

第2回演習の解説

- 第2回ポイント資料の裏面（演習1・演習2・確認問題）の正解を解説する
- 解説は，演習の締切（講義開始30分後）のあとに行う
- 解答例は電子的には配布しない．**自分の解答に転記すること**（解説中の撮影は不可）
  - 講義中に自分の解答へ書き足す・訂正するのはよい．そのまま試験に持ち込める
- 提出がまだの人は，WebClass の「【締切後】第2回 演習」から提出する（0.7点で評価）

今回のトピック

1. リレーショナルデータモデルの制約
    - キー，外部キー制約，空値（NULL）
2. その他の一貫性制約とSQLによる制約記述
3. 今回の演習と小テストの「型」

教科書：第3章 p.25〜33

Topic 1

リレーショナルデータモデルの制約

トピックのキーワード

- 候補キー・主キー・キー制約
- 外部キー・外部キー制約（参照整合性制約）
- 空値（NULL）・NOT NULL制約

データモデルの制約（constraint）とは

データベース内のデータの一貫性を保つために，データが守るべきルール

- 一貫性を保つ＝矛盾したデータを生じないようにする
  - 例）同じ学籍番号を持つ学生が2人いてはいけない
  - 例）学籍番号は必ず数字でなければならない
- 矛盾したデータが入るとなかなか気づけないからチェックは重要
- データに関する共通的なチェックはアプリケーションではなく，データベース側で行う
- 「一貫性制約」「整合性制約」とも呼ばれる
- 注：教科書では制約ではなく，「意味記述」という用語が使われている．

リレーショナルデータモデルにおける制約

- ドメイン制約：値の範囲
- キー制約：タプルの一意性
- 外部キー制約（参照整合性制約）
- 空値（NULL）に関する制約
- 上記の他に「関数従属性」もリレーショナルデータモデルにおける制約の一つとする考え方もあるが，これは，リレーショナルデータモデルの設計・正規化の中で説明する

ドメイン制約

復習(リレーションの集合的な定義で述べた)

- ドメイン（定義域とも呼ぶ）：属性のとりうる値を集めた集合
  - 例1：文字列全体，数値全体，真偽値
  - 例2：最大長10の文字列全体
  - 例3：人名の集合，給与の集合，年齢の集合

ドメイン制約とは，「属性の値は，必ずドメインに含まれなければならない」である

- 例えば，上記の例2のドメインを持つ属性は，11文字の値を属性の値に入れてはいけない
- DBMSの動作としては，値を挿入しようとしたときにエラーにすべき
  →一貫性（整合性）が守られる

キー制約

リレーションにおけるキーとは，（雑に言うと）「タプルを一意に識別する属性」のこと

「タプルを一意に識別する」ことをもう少し掘り下げてみる

注意：タプルを一意に識別する制約を，**UNIQUE制約**と呼ぶ（キー制約の一部）

- 下のリレーション 社員（教科書 図3.1）のタプルを一意に識別する属性は何か？

社員

社員番号	社員名	給与	所属	健保番号
0650	山田太郎	50	K55	80596
1508	鈴木花子	40	K41	81403
0231	田中桃子	60	K41	80201
2034	佐藤一郎	40	K55	81998

タプルを一意に識別する（UNIQUE制約）

- 社員番号：一意に識別できそうだ（社員番号が0650の社員は一人しかいない）
- 社員名：今はすべての社員名が異なっているが，同姓同名の社員が入社してくるかもしれない．よって，一意に識別できない
- 健保番号：異なる社員が同じ健保番号をもたないから，一意に識別できる

社員

社員番号	社員名	給与	所属	健保番号
0650	山田太郎	50	K55	80596
1508	鈴木花子	40	K41	81403
0231	田中桃子	60	K41	80201
2034	佐藤一郎	40	K55	81998

タプルを一意に識別する

前ページの例の大切なポイント

1. 属性によりタプルが一意に識別できるどうかは，そのリレーションのデータがどうなっているかで決まるのではなく，属性の意味で決まる

- 現在のリレーション 社員では，社員名はデータとしては一意に識別できるが，属性の意味から考えると，同姓同名がありえるから，タプルを一意に識別できるとは言えない

2. タプルを一意に識別する属性は，一つとは限らない

- 社員番号と健保番号は両方ともタプルを一意に識別できる

タプルを一意に識別する

- 下のリレーション 納品のタプルを一意に識別する属性は何か？
  - 最初のタプルは，商品番号 G1の商品を，顧客番号C1の顧客に3個納品したことを意味する
  - 教科書 図3.2 のリレーション

納品

商品番号	顧客番号	納品数量
G1	C1	3
G1	C2	10
G2	C2	5
G2	C3	10

タプルを一意に識別する

- 商品番号だけ，顧客番号だけでは，タプルを一意に識別できない
- 商品番号，顧客番号，納品数量のすべての属性を用いるとタプルを一意に識別できる
- しかし，商品番号と顧客番号の組であれば，その納品数量が決まると考えると，その2つの属性によりタプルを一意に識別できる
  - 注：一般的には顧客に対してある商品を同じ数量だけ2回納品する場合もあるだろう．そのときは，納品イベントを表す識別子で一意に識別できるだろう

納品

商品番号	顧客番号	納品数量
G1	C1	3
G1	C2	10
G2	C2	5
G2	C3	10

タプルを一意に識別する

前ページの例の大切なポイント

3. 複数の属性の組により，タプルを一意に識別できることもある

4. タプルを一意に識別する属性の組があるときに，それに別の属性を付け加えた属性の組でもタプルを一意に識別できる

- 前の例だと，「商品番号」と「顧客番号」だけで一意に識別できるが，それに「納品数量」を加えた３つの属性によっても一意に識別できる
- でも，我々にとって意味があるのは「極小な」属性群
  - 極小：一つでも属性を除くと一意でなくなる

候補キー（candidate key）

これまで述べたポイントを元にして，キーに関する用語を整理する

その値を指定すると，リレーションのタプルを一意に識別できる属性の集合であって極小であるものを「候補キー」と呼ぶ

- リレーション 社員の候補キーは，{社員番号},{健保番号}
- リレーション 納品の候補キーは，{商品番号, 顧客番号}

社員

社員番号	社員名	給与	所属	健保番号
0650	山田太郎	50	K55	80596
1508	鈴木花子	40	K41	81403
0231	田中桃子	60	K41	80201
2034	佐藤一郎	40	K55	81998

候補キー　　候補キー

納品

商品番号	顧客番号	納品数量
G1	C1	3
G1	C2	10
G2	C2	5
G2	C3	10

候補キー（2つの属性の組）

主キー（primary key）

候補キーの中から，データベース設計者がデータ管理上最も適当であり，かつその属性値は空値になりえないものを選択したものを「主キー」と呼ぶ

- 主キーは，設計者の主観によって選ばれる（空値については，後で説明する）
- 社員の主キーは，（普通なら）{社員番号}．健保番号を主キーにしても絶対に間違いとは言えない
- 納品の主キーは，{商品番号, 顧客番号}

社員

社員番号	社員名	給与	所属	健保番号
0650	山田太郎	50	K55	80596
1508	鈴木花子	40	K41	81403
0231	田中桃子	60	K41	80201
2034	佐藤一郎	40	K55	81998

主キー　候補キー

納品

商品番号	顧客番号	納品数量
G1	C1	3
G1	C2	10
G2	C2	5
G2	C3	10

主キー

キー制約

キー制約とは，以下の性質を満たすことを言う

主キーは次の条件を満たさなければならない

1. **主キーはタプルを一意に識別できること**
2. **主キーを構成する属性は空値をとらないこと**

- これらは，主キーの定義そのものである

主キーとなる属性を追加する例

- リレーション 納品では，商品番号・顧客番号いずれでもタプルを識別できないので，{商品番号, 顧客番号}という集合を主キーにする必要があった

納品

商品番号	顧客番号	納品数量
G1	C1	3
G1	C2	10
G2	C2	5
G2	C3	10

このような場合は，「納品番号」という主キーとなる属性を人工的に作って，それを主キーとすることがある

- 一つの属性だけの主キーはシンプルで扱いやすい
- 意味的に納品を識別していることがわかりやすい

主キーを含めた識別性の設計は重要

- データベースの設計においては，前ページのように主キーとなる属性を追加することも含めて，どのようにデータを識別するかを注意深く設計する必要がある
- 実際には...
  - システム全体でのデータの識別が問題になることが多い
  - 銀行のお客様の口座は口座番号で識別できる←主キー
  - お客様はどうやって識別する？
    - 口座番号：複数の口座を持っている人がたくさんいる
  - お客様の家族はどうやって識別する？
    - 住所と姓だけでOKか？
- データベースの設計ではこのような要件（要求条件）を考えてリレーションスキーマの設計、キーの割り当てを行っていかなければならない

外部キー

- データベースに含まれる複数のリレーションの中に，「同じ意味」を表すデータが含まれる場合がある
- 下の例は，商品を表すリレーション（主キーは商品コード）と，その商品がいくつ販売されたのかを表すリレーション（主キーは販売通番）である
- 両方のリレーションの「商品」は，同じ意味を表している
    - 伝票明細の1010番の商品コードを持つ商品は，商品Aである

商品

商品コード	商品名	単価
1010	商品A	2500
1020	商品B	1750
1033	商品C	3200
1052	商品D	180
2020	商品E	950

伝票明細

販売通番	商品コード	販売個数
0001	1010	1
0002	1052	10
0003	1033	1
0004	1020	2
0005	2020	5

外部キー

- 伝票明細の商品コードの値は，必ず商品の商品コードに存在しないと矛盾する（存在しない商品を販売したことになる）
- 他のリレーションの主キーを参照している側の属性を「**外部キー**」と呼ぶ
- 定義：「**その値が，空値であるか，そうでなければ，必ずある特定のリレーションの主キーのどれかの値と一致する属性**」



商品コード	商品名	単価
1010	商品A	2500
1020	商品B	1750
1033	商品C	3200
1052	商品D	180
2020	商品E	950

販売通番	商品コード	販売個数
0001	1010	1
0002	1052	10
0003	1033	1
0004	1020	2
0005	2020	5

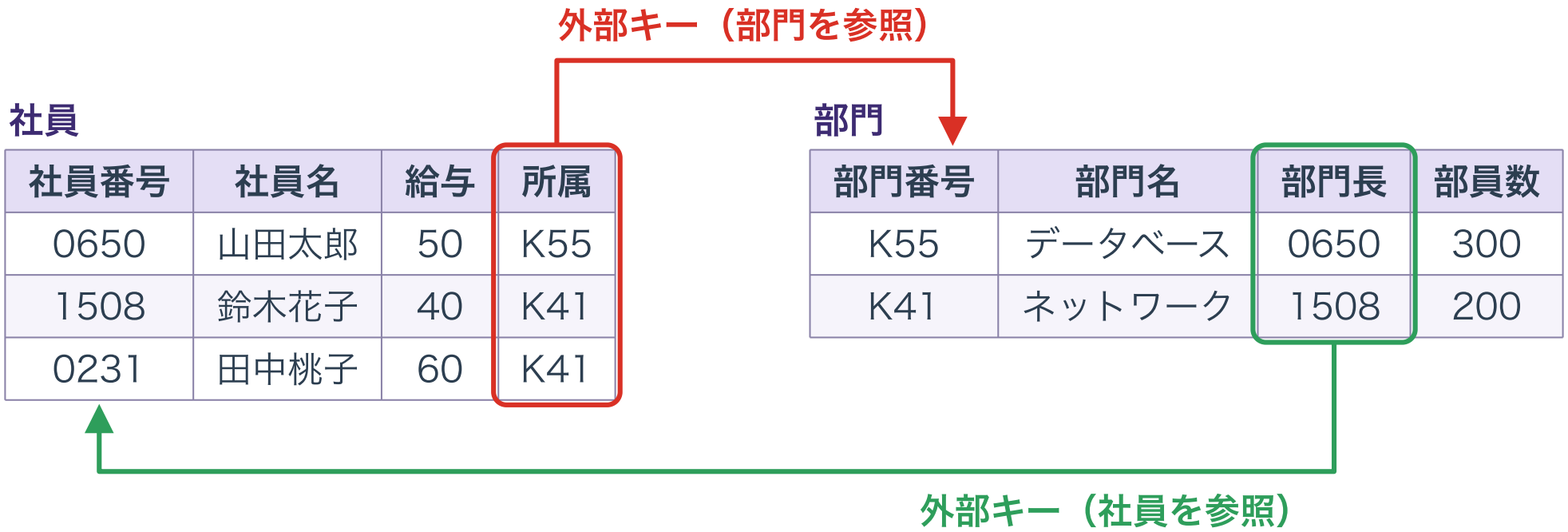
外部キー制約

外部キーが指定されたときに，いかなるタプルも外部キーの定義に述べた条件を満たさなければならない，という制約を「外部キー制約」，または，「参照整合性制約」と呼ぶ

- 前の例では，外部キーは一つの属性だったが，主キーが複数の属性の集合からなる場合もあるから，複数の属性の集合からなる外部キーもありえる
  - 例）組織を部番号＋課番号で識別するとき

外部キー：教科書の例

- 教科書 図3.3 の社員–部門データベース．2つのリレーションが互いに参照している
  - 社員の「所属」は，部門の「部門番号」を参照する外部キー
  - 部門の「部門長」は，社員の「社員番号」を参照する外部キー



外部キー制約が守られていない例

- 伝票明細の商品コード 1052 は，商品の主キーのどの値とも一致しない



外部キー制約によって守られること

1. リレーション 商品にない商品を伝票明細に書いてはいけない
2. 伝票明細にデータがある商品をリレーション 商品から削除してはいけない



商品コード	商品名	単価
1010	商品A	2500
1020	商品B	1750
1033	商品C	3200
1052	商品D	180
2020	商品E	950

販売通番	商品コード	販売個数
0001	1010	1
0002	1052	10
0003	1033	1
0004	1020	2
0005	2020	5

空値（NULL）

- 今までは「値」がとにかく入っていた
  - 例）個数が，1，10，商品名は，商品A，商品B
- 「値が決まっていない」、「値がない」というのもある
  - FAXがない人には，属性「FAX番号」の値はない
  - 社長には，リレーション 人事情報の属性 上司の値はない
  - 平社員には，属性 部下 の値はない
- これらのデータがない状態を表す特殊な属性値が「**空値（NULL）**」である
  - NULLは「ナル」と発音する．

NULLは「普通の値」ではない

- NULL同士は比較できない
  - FAXがない人同士でFAX番号が同じかどうかは意味をなさない
  - 等しいかどうかだけでなく、大小関係もない
- SQL言語の中でもNULL（大文字）
- 「空」とも呼ぶ．教科書では「空」推奨だが，「空値」のほうが一般的だろう

NOT NULL制約

- 属性は必ずどれかの値をとり，NULLにはならないという制約
  - 姓名のない人はいないから，姓と名はNOT NULL制約を満たす
  - Middle Nameのない人はいるから，これはNOT NULL制約は満たさない

学籍番号・姓・名などは必ず値を持つ：NOT NULL

学生

学籍番号	姓	Middle Name	名	学部	入学年度
00001	秋吉	Mary	久美子	情報	2024
00002	赤坂	George	泰彦	情報	2024
00003	飯島	Samantha	愛	情報	2025
00004	泉		ピン子	情報	2025

Middle Name のない人もいる：NOT NULL ではない

Topic 2

その他の一貫性制約とSQLによる制約記述

トピックのキーワード

- CREATE TABLE
- 検査制約・表明・トリガ

SQLによる制約記述（CREATE TABLE）

- 実際のDBMSでは，制約はテーブルを作成するときに定義する
- テーブルを作成するには，CREATE TABLEコマンドを使う
- 下の例は，部署テーブルの例で以下の制約が定義されている
  - 部署IDは，整数型で主キーである
  - 部署名は，テキスト型でユニークでNULL不可である
- 以下の例は，Google Colab（SQLite）で動かせる．ノートブックを学生ポータルで共有する

```
CREATE TABLE department (
    dept_id     INTEGER PRIMARY KEY,
    dept_name   TEXT UNIQUE NOT NULL
);
```

SQLによる制約記述

- 以下の例にはこれまで述べていない，検査制約が含まれる

```
CREATE TABLE employee (
    emp_id      INTEGER PRIMARY KEY,                    -- 主キー制約
    name        TEXT NOT NULL,                          -- NOT NULL制約
    email       TEXT UNIQUE,                            -- UNIQUE制約
    salary      REAL CHECK (salary >= 0),               -- 検査制約
    dept_id     INTEGER REFERENCES department(dept_id), -- 外部キー制約
    age         INTEGER CHECK (age BETWEEN 18 AND 65)   -- 検査制約
);
```

Google Colab の使い方（1）開き方

- Google Colab：ブラウザだけで Python や SQL を動かせるサービス．インストール不要
- 開き方
    i. Google アカウントにログインした状態で，学生ポータルの第3回ページのリンクを開く
    ii. 「Google が作成したものではありません」と出たら「このまま実行」
    iii. 自分用に残すなら「ファイル → ドライブにコピーを保存」（保存しなくても動く）

Google Colab の使い方（2）基本操作

- セルの実行：セルをクリックして **Shift + Enter**（またはセル左の再生ボタン）
- セルは**上から順に**実行する．最初の環境作成のセルを飛ばすとエラーになる
- しばらく放置すると切断される．「ランタイム → ランタイムを再起動」のあと上から再実行
- 使い方はノートブックの冒頭にも書いてある

制約違反を体験する（Google Colab）

学生ポータルの第3回のノートブックで，制約に違反するデータを入れてみる．**エラーになるのが正しい動作**

試すこと	違反する制約
同じ emp_id の従業員を登録する	主キー制約
name を NULL にして登録する	NOT NULL制約
同じ email の従業員を登録する	UNIQUE制約
salary をマイナスにして登録する	検査制約
存在しない dept_id（99）の従業員を登録する	外部キー制約

- 講義中に実演する．各自でも動かしてみること（演習の提出には関係しない）

より高度な制約（CHECK・DOMAIN）

- 検査制約は，あるカラムの値が満たすべき条件を，事前に定義しておいて検査できる仕組み
  - 例）学部の名前は，情報か事業創造かアニメ・マンガのいずれか
  - 例）社員の給料は必ず，プラスになる
  - 検査を満たさないとデータの挿入が失敗する
- ドメイン制約は，CREATE DOMAINというSQL構文があり，PostgreSQLでは利用することが可能（MySQL・SQLiteでは未対応）

表明

- 検査制約は，レコード毎に検査する仕組み
- 一方，テーブル全体の制約や，複数のテーブルにまたがるような，複雑な制約を「表明（assertion）」と呼ぶ
  - SQL標準には規定されている
  - 実際のDBMSにはほとんど実装されていない（参考扱い）
- 以下の例は，口座の「合計」残高が負にならない制約を表す

```
CREATE ASSERTION total_balance_nonnegative
CHECK (
    (SELECT SUM(balance) FROM account) >= 0
);
```

トリガ

- あるテーブルへの挿入などを契機に他のテーブルに必要な更新などを施す仕組み
- データベースの一貫性の保証にも活用することもできる
- 以下は，給料が更新されたら，その変更履歴をログとして保存する例

```
CREATE TRIGGER log_salary_update
AFTER UPDATE OF salary ON employee
FOR EACH ROW
WHEN OLD.salary <> NEW.salary
BEGIN
    INSERT INTO salary_log VALUES (
        OLD.emp_id,
        OLD.salary,
        NEW.salary,
        datetime('now')
    );
END;
```

Topic 3

今回の演習と小テストの「型」

制約の種類のまとめ

制約	内容	SQLでの書き方
ドメイン制約	属性の値は決められた値の集合に含まれる	データ型
キー制約	主キーは一意で，空値をとらない	PRIMARY KEY
UNIQUE制約	値が重複しない	UNIQUE
外部キー制約	参照先の主キーにある値か空値	REFERENCES
NOT NULL制約	空値にならない	NOT NULL
検査制約	値が条件を満たす	CHECK

今回の小テストの「型」

- **型D：キーの指摘**　リレーションスキーマを見て，候補キー・主キー・外部キーを答える
  - 例）社員 (社員番号, 社員名, 給与, 所属, 健保番号)，部門 (部門番号, 部門名, 部門長, 部員数)
  - 社員の候補キー：{社員番号}，{健保番号}　主キー：{社員番号}
  - 外部キー：社員の「所属」→ 部門の「部門番号」，部門の「部門長」→ 社員の「社員番号」
- **型E：制約の例**　制約を1つ，「制約の種類：内容」の形で書く
  - 例）NOT NULL制約：社員の「社員名」は空値にならない
- 外部キーは「どのリレーションのどの属性を参照するか」まで書く

第3回演習と小テスト1

- ポイント資料の裏面を，表面の例にならって自分の手で解く
  - 演習1：型Dの類題　演習2：型Eの類題　演習3：型Aの類題（5題）　確認問題（2題）
- 提出：表面・裏面をそれぞれ撮影して，WebClass「第3回 演習」へ
  - 締切：**次回（10/21）の講義開始30分後**
- **小テスト1：第5回（10/28）の講義の最後15分**
  - 範囲：第1〜3回（今回まで）．ポイント資料の例・演習と同じ型の問題を出す
  - 持ち込み：第1〜3回のポイント資料（3枚）のみ
- 次回の予習：教科書 第4章 p.34〜47（リレーショナルデータモデル：操作記述）